

Feature Descriptor Matching, Matching Refinement, and RANSAC

Teaching Notes for Computer Vision

1 Motivation: From Feature Detection to Geometric Correspondence

A local feature pipeline usually has the following structure:

Image $\rightarrow$ Keypoint Detection $\rightarrow$ Descriptor $\rightarrow$ Descriptor Matching $\rightarrow$ Match Filtering $\rightarrow$ Geometric Verification

A detector such as Harris, SIFT, ORB, or another feature detector identifies salient image locations. A descriptor represents the local appearance around each keypoint. Matching attempts to establish correspondences between descriptors in two images.

However, a descriptor match is only a *candidate correspondence*. Two visually similar local patches can occur at unrelated locations. Therefore, a practical system generally requires:

$$\boxed{\text{Descriptor similarity} + \text{Geometric consistency}}$$

rather than descriptor similarity alone.

This document discusses:

1. Brute-Force (BF) descriptor matching,
2. FLANN-based approximate nearest-neighbour matching,
3. nearest-neighbour, cross-check, ratio and group tests,
4. RANSAC-based geometric verification,
5. practical implementation considerations,
6. application-specific requirements for accuracy, latency, and hardware.

2 Descriptor Matching as a Nearest-Neighbour Problem

Let the first image contain descriptors

$$\mathcal{D}_A = \{\mathbf{d}_1^A, \mathbf{d}_2^A, \dots, \mathbf{d}_N^A\}$$

and the second image contain

$$\mathcal{D}_B = \{\mathbf{d}_1^B, \mathbf{d}_2^B, \dots, \mathbf{d}_M^B\}.$$

For a descriptor $\mathbf{d}_i^A$, the matching problem is to find

$$j^* = \arg \min_j D(\mathbf{d}_i^A, \mathbf{d}_j^B),$$

where $D(\cdot, \cdot)$ is an appropriate descriptor distance.

For real-valued descriptors such as SIFT, a common choice is Euclidean distance:

$$D_2(\mathbf{a}, \mathbf{b}) = \|\mathbf{a} - \mathbf{b}\|_2 = \sqrt{\sum_{k=1}^d (a_k - b_k)^2}.$$

For binary descriptors such as BRIEF and ORB, Hamming distance is appropriate:

$$D_H(\mathbf{a}, \mathbf{b}) = \sum_{k=1}^d \mathbf{1}[a_k \neq b_k].$$

Equivalently, for binary vectors,

$$D_H(\mathbf{a}, \mathbf{b}) = \text{popcount}(\mathbf{a} \oplus \mathbf{b}).$$

The distance must match the descriptor representation. OpenCV recommends L1/L2 norms for floating-point descriptors such as SIFT and Hamming distance for binary descriptors such as ORB and BRIEF; for ORB with $WTA_K = 3$ or 4 , `NORM_HAMMING2` is appropriate [1].

3 Brute-Force Matcher

3.1 Basic Idea

The Brute-Force (BF) matcher performs an exhaustive nearest-neighbour search.

For every descriptor in image A , it computes its distance to every descriptor in image B and chooses the smallest distance:

$$j^*(i) = \arg \min_{j \in \{1, \dots, M\}} D(\mathbf{d}_i^A, \mathbf{d}_j^B).$$

The procedure is therefore:

1. Select one descriptor from image A .
2. Compute its distance to every descriptor in image B .
3. Select the descriptor with minimum distance.
4. Repeat for every descriptor in image A .

3.2 Computational Complexity

For N descriptors in image A , M descriptors in image B , and descriptor dimensionality d :

$$\boxed{O(NMd)}$$

distance computations are required in the straightforward implementation.

For SIFT, $d = 128$. If both images contain 2000 descriptors, the matcher may evaluate approximately

$$2000 \times 2000 = 4 \times 10^6$$

descriptor comparisons, each involving a 128-dimensional distance.

BF matching is therefore attractive when:

- the descriptor sets are relatively small,
- exact nearest neighbours are desirable,
- the application prioritizes matching accuracy over large-scale search speed,
- binary descriptors can exploit very fast Hamming/popcount operations.

3.3 BF Matching Pseudocode

```
Input:
  Descriptor sets DA and DB
  Distance function D

For each descriptor di in DA:
  best_distance = infinity
  best_index = -1

  For each descriptor dj in DB:
    distance = D(di, dj)

    If distance < best_distance:
      best_distance = distance
      best_index = j

  Store match (i, best_index, best_distance)

Return all matches
```

3.4 OpenCV Implementation

Typical Python usage is:

```
# SIFT
sift = cv2.SIFT_create()
kp1, des1 = sift.detectAndCompute(img1, None)
kp2, des2 = sift.detectAndCompute(img2, None)

bf = cv2.BFMatcher(cv2.NORM_L2, crossCheck=False)
matches = bf.match(des1, des2)
```

For ORB:

```
orb = cv2.ORB_create()
kp1, des1 = orb.detectAndCompute(img1, None)
kp2, des2 = orb.detectAndCompute(img2, None)

bf = cv2.BFMatcher(cv2.NORM_HAMMING, crossCheck=False)
matches = bf.match(des1, des2)
```

OpenCV's `DMatch` object provides, among other fields, `queryIdx`, `trainIdx`, and `distance` [1].

4 Cross-Check Matching

A one-way nearest neighbour does not guarantee a reciprocal match.

Suppose

$$A_i \rightarrow B_j$$

is the nearest match from image A to image B . It may still be true that

$$B_j \rightarrow A_k, \quad k \neq i.$$

A cross-check retains the pair only if:

$$A_i \rightarrow B_j \quad \text{and} \quad B_j \rightarrow A_i$$

Formally,

$$j^*(i) = \arg \min_j D(A_i, B_j)$$

and

$$i^*(j) = \arg \min_i D(B_j, A_i).$$

The match (i, j) is retained when

$$\boxed{j^*(i) = j \quad \wedge \quad i^*(j) = i.}$$

Cross-checking removes many asymmetric or ambiguous matches, but it can also discard valid correspondences when several descriptors are genuinely similar.

OpenCV exposes this using `crossCheck=True`; its documentation describes cross-checking as a consistent reciprocal matching criterion and as an alternative to Lowe’s ratio test [1].

5 The Ratio Test

5.1 Motivation

Suppose the nearest two candidate descriptors for $\mathbf{d}_i^A$ have distances

$$d_1 < d_2.$$

A small d_1 alone does not necessarily indicate a reliable match. If d_2 is almost equal to d_1 , the descriptor has two nearly equally good candidates and is therefore ambiguous.

The ratio test uses:

$$\boxed{r = \frac{d_1}{d_2}.}$$

The match is accepted if

$$\boxed{\frac{d_1}{d_2} < \tau}$$

where τ is a chosen threshold.

Typical practical values include approximately 0.7–0.8, but the best value depends on the descriptor, image conditions, and desired precision/recall trade-off. OpenCV examples commonly demonstrate values such as 0.7 or 0.75 [1].

5.2 Interpretation

If

$$d_1 = 0.20, \quad d_2 = 0.80,$$

then

$$r = 0.25,$$

which indicates a strongly preferred nearest neighbour.

If

$$d_1 = 0.45, \quad d_2 = 0.50,$$

then

$$r = 0.90,$$

which indicates ambiguity.

Therefore:

$$\boxed{\text{small ratio} \Rightarrow \text{well-discriminated match}}$$

and

$$\boxed{\text{ratio near 1} \Rightarrow \text{ambiguous match.}}$$

5.3 Algorithm

```
For each descriptor di in image A:

    Find its two nearest neighbours:
        m1 = nearest descriptor
        m2 = second-nearest descriptor

    r = distance(m1) / distance(m2)

    If r < tau:
        accept m1
    Else:
        reject m1
```

In OpenCV:

```
matches = bf.knnMatch(des1, des2, k=2)

good = []
for m, n in matches:
    if m.distance < 0.75 * n.distance:
        good.append(m)
```

6 Group or Mutual-Consistency Tests

The phrase “group test” is used in several ways in feature-matching systems. A useful teaching interpretation is to consider a match in relation to its neighbouring candidates rather than judging it only from one descriptor pair.

A simple form is a *mutual nearest-neighbour group test*: retain a candidate only when the correspondence is supported consistently from both descriptor sets.

A stronger practical strategy is to combine several tests:

Candidate = nearest neighbour + ratio test + mutual consistency + geometric consistency.

For example:

$$j^*(i) = \arg \min_j D(A_i, B_j),$$

$$i^*(j) = \arg \min_i D(B_j, A_i),$$

$$\frac{d_1(i)}{d_2(i)} < \tau,$$

$$\text{geometric residual} < \epsilon.$$

Only correspondences satisfying the required conditions are retained.

6.1 Caveat

Descriptor-space filtering and geometric filtering solve different problems.

The ratio test asks:

“Is this descriptor significantly more similar to one candidate than to alternatives?”

RANSAC asks:

“Does this correspondence agree with a common geometric transformation?”

Therefore, a ratio test should not be regarded as a substitute for geometric verification.

7 FLANN Matcher

7.1 Motivation

BF performs exhaustive nearest-neighbour search. This is accurate but can become expensive when:

$$N, M \gg 1.$$

FLANN stands for *Fast Library for Approximate Nearest Neighbors*. It provides data structures and search procedures designed to accelerate nearest-neighbour queries, particularly for larger and higher-dimensional datasets [2].

The central trade-off is:

$$\text{speed} \longleftrightarrow \text{exactness}.$$

Unlike BF, FLANN generally performs an *approximate* nearest-neighbour search.

7.2 KD-Tree Intuition

For real-valued descriptors such as SIFT, a KD-tree can recursively partition feature space.

In a simplified two-dimensional example:



The tree recursively partitions the space using coordinate-based splits.

For 128-dimensional SIFT descriptors, the same idea operates in a much higher-dimensional space.

FLANN can build multiple randomized trees and search only selected portions of the structure. OpenCV examples for SIFT commonly use a KD-tree configuration with multiple trees and a configurable number of search checks [2].

7.3 Approximate Search

Let the exact nearest neighbour be

$$j^* = \arg \min_j D(\mathbf{q}, \mathbf{d}_j).$$

An approximate search returns

$$\hat{j}$$

such that

$$D(\mathbf{q}, \mathbf{d}_{\hat{j}})$$

is hopefully close to the exact minimum.

The parameter controlling search effort affects the probability of recovering the true nearest neighbour.

More search effort:

$\Rightarrow$ higher accuracy, higher computational cost.

Less search effort:

$\Rightarrow$ lower cost, potentially more approximate matches.

7.4 OpenCV Example

```
sift = cv2.SIFT_create()

kp1, des1 = sift.detectAndCompute(img1, None)
kp2, des2 = sift.detectAndCompute(img2, None)

index_params = dict(
    algorithm=cv2.FLANN_INDEX_KDTREE,
    trees=5
)

search_params = dict(checks=50)

flann = cv2.FlannBasedMatcher(
    index_params,
    search_params
)

matches = flann.knnMatch(
    des1, des2, k=2
)
```

OpenCV documents the use of KD-tree based FLANN parameters for floating-point descriptors such as SIFT and the `checks` parameter for controlling search effort [2].

7.5 Important FLANN Caveat for Binary Descriptors

A common mistake is to assume that “FLANN is always appropriate for ORB.”

FLANN configurations intended for floating-point descriptors such as SIFT are not the same as the appropriate search structures for binary descriptors.

For binary descriptors, a Hamming-compatible matcher or an appropriate binary nearest-neighbour index should be used. In OpenCV, BF with `NORM_HAMMING` is often the simplest and reliable choice for ORB/BRIEF/BRISK [1].

8 BF versus FLANN

Property	BF Matcher	FLANN Matcher
Search	Exhaustive	Approximate nearest neighbour
Accuracy	Exact nearest neighbour for chosen metric	Approximate; depends on index/search parameters
Complexity	Approximately $O(NMd)$	Data-structure dependent; typically much faster for large searches
SIFT	Excellent	Excellent for large descriptor sets
ORB/BRIEF	Hamming distance	Requires binary-compatible index/configuration
Small descriptor sets	Often preferable	Often unnecessary
Large descriptor sets	Can become expensive	Usually more attractive
Parameter tuning	Minimal	Index and search parameters matter
Determinism	Generally straightforward	Can depend on indexing/search configuration

The important engineering principle is:

Do not choose FLANN simply because it is called “fast”.

Benchmark the complete pipeline on the target hardware and descriptor set.

9 From Candidate Matches to Geometrically Valid Matches

After descriptor matching, suppose we have correspondences:

$$\mathcal{C} = \{(\mathbf{x}_i, \mathbf{x}'_i)\}_{i=1}^N.$$

Some correspondences are correct:

$$\mathcal{I} \subset \mathcal{C},$$

while others are outliers:

$$\mathcal{O} = \mathcal{C} \setminus \mathcal{I}.$$

The goal of geometric verification is to estimate a model that explains the inlier correspondences while rejecting outliers.

Examples include:

- affine transformation,
- homography,
- fundamental matrix,
- essential matrix,
- rigid transformation,
- similarity transformation.

RANSAC is one of the most important classical tools for this purpose.

10 RANSAC

RANSAC stands for *Random Sample Consensus*. It estimates a model in the presence of a substantial number of outliers.

The central idea is surprisingly simple:

Randomly choose a minimal subset $\rightarrow$ fit a model $\rightarrow$ count supporting points $\rightarrow$ repeat.

10.1 Example: Homography Estimation

For planar scenes or pure camera rotation, a homography satisfies

$$\mathbf{x}' \sim H\mathbf{x},$$

where

$$H \in \mathbb{R}^{3 \times 3}.$$

A homography has eight independent degrees of freedom because scale is arbitrary.

The minimum number of point correspondences required is therefore four.

For four corresponding points:

$$\mathbf{x}_i \leftrightarrow \mathbf{x}'_i, \quad i = 1, \dots, 4,$$

we can estimate H .

10.2 RANSAC Homography Algorithm

```
Input:
  Candidate correspondences C
  Minimal sample size s = 4
  Number of iterations N
  Geometric threshold epsilon

best_model = None
best_inliers = empty set

Repeat N times:

  1. Randomly select 4 correspondences.

  2. Estimate homography H from them.

  3. For every correspondence (xi, xi'):

    Predict:
      x_hat_i' ~ H xi

    Compute geometric error.

    If error < epsilon:
      classify as INLIER
    else:
      classify as OUTLIER

  4. Count inliers.

  5. If current model has more inliers
    than best_model:

    best_model = H
    best_inliers = current inliers

Optionally:
  Re-estimate H using all best inliers.

Return:
  H
  inlier set
```

11 Geometric Error in RANSAC

A simple reprojection error is:

$$e_i = \|\mathbf{x}'_i - \pi(H\mathbf{x}_i)\|_2,$$

where $\pi(\cdot)$ converts homogeneous coordinates back to Euclidean coordinates.

The point is considered an inlier if

$$e_i < \epsilon.$$

Here ϵ is a critical parameter.

Small ϵ :

$\Rightarrow$ high precision but potentially fewer inliers.

Large ϵ :

$\Rightarrow$ more inliers but greater risk of accepting incorrect matches.

The threshold should reflect the expected image noise, localization error, and application requirements.

11.1 Symmetric Transfer Error

For a homography, a stronger criterion considers both directions:

$$e_i^{\text{sym}} = \|\mathbf{x}'_i - \pi(H\mathbf{x}_i)\|_2^2 + \|\mathbf{x}_i - \pi(H^{-1}\mathbf{x}'_i)\|_2^2.$$

This reduces the dependence on only one mapping direction.

In practical implementations, the exact error metric depends on the estimator and library.

12 How Many RANSAC Iterations?

Suppose:

- p = desired probability of obtaining at least one all-inlier sample,
- w = probability that a randomly selected correspondence is an inlier,
- s = number of points required to estimate the model,
- N = number of RANSAC iterations.

The probability that one random sample consists entirely of inliers is

$$w^s.$$

The probability that it contains at least one outlier is

$$1 - w^s.$$

After N independent iterations:

$$(1 - w^s)^N$$

is the probability that every sample contains at least one outlier.

Therefore:

$$1 - (1 - w^s)^N = p.$$

Solving for N :

$$N = \frac{\log(1 - p)}{\log(1 - w^s)}$$

For example, if

$$w = 0.5, \quad s = 4, \quad p = 0.99,$$

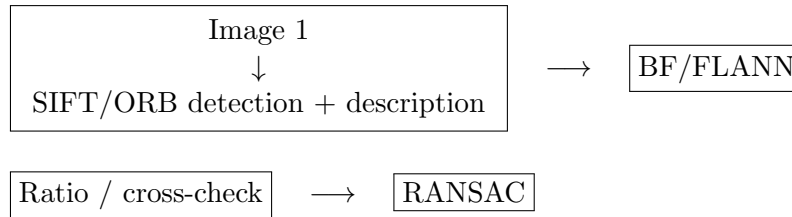
then approximately

$$N \approx 72.$$

If the inlier ratio decreases substantially, the required number of iterations increases rapidly.

13 RANSAC with Feature Matching

A complete classical matching pipeline can therefore be:



More explicitly:

Descriptors $\rightarrow$ Nearest-neighbour candidates $\rightarrow$ Ambiguity filtering
 $\rightarrow$ Geometric verification $\rightarrow$ Verified correspondences.

This distinction is fundamental:

descriptor matching $\neq$ geometrically valid matching.

14 Practical OpenCV RANSAC Example

For planar image matching:

```
import numpy as np
import cv2

src_pts = np.float32([
    kp1[m.queryIdx].pt for m in good_matches
]).reshape(-1, 1, 2)

dst_pts = np.float32([
    kp2[m.trainIdx].pt for m in good_matches
]).reshape(-1, 1, 2)

H, mask = cv2.findHomography(
    src_pts,
    dst_pts,
    cv2.RANSAC,
    5.0
)

inliers = [
    m for m, keep in zip(good_matches, mask.ravel())
    if keep
]
```

The threshold of 5.0 pixels shown above is only an example. It should not be interpreted as a universal value.

15 Important RANSAC Caveats

15.1 The Geometric Model Must Be Correct

RANSAC does not decide which model is physically correct.

If the scene is approximately planar:

$$\mathbf{x}' \sim H\mathbf{x}$$

is appropriate.

For general 3D scenes observed by two calibrated or uncalibrated cameras, the fundamental matrix may be more appropriate:

$$\mathbf{x}'^T F \mathbf{x} = 0.$$

For calibrated cameras:

$$\mathbf{x}'^T E \mathbf{x} = 0.$$

Using a homography on a general non-planar scene can produce misleading results.

15.2 Degenerate Samples

Some random samples cannot determine a valid model.

For example, four nearly collinear points are problematic for homography estimation.

Good RANSAC implementations therefore need to detect or avoid degenerate samples.

15.3 Threshold Selection

The RANSAC threshold should account for:

- keypoint localization uncertainty,
- image noise,
- lens distortion,
- image resolution,
- descriptor/keypoint quality,
- expected geometric model error.

A threshold that is suitable for a 640×480 image may not be appropriate for a 4K image.

15.4 RANSAC Is Not a Descriptor Matcher

RANSAC cannot rescue an arbitrarily bad descriptor matcher.

If there are too few correct correspondences, or if the inlier ratio is extremely low, RANSAC may fail.

The practical pipeline is therefore:

Good descriptors $\rightarrow$ good candidate matches $\rightarrow$ RANSAC refinement.

16 Integrated Matching Algorithm

The following pseudocode captures a robust classical feature matching pipeline.

```
INPUT:
  Images I1, I2

1. Detect keypoints
   K1 = Detector(I1)
   K2 = Detector(I2)
```

```

2. Compute descriptors
   D1 = Descriptor(I1, K1)
   D2 = Descriptor(I2, K2)

3. Descriptor matching
   M = BF or FLANN(D1, D2)

4. Candidate filtering
   Apply distance threshold
   and/or ratio test

5. Optional reciprocal consistency
   Apply cross-check / mutual matching

6. Geometric verification
   Select geometric model:
       affine / homography /
       fundamental matrix / essential matrix

7. RANSAC
   Randomly select minimal samples
   Estimate model
   Compute geometric residuals
   Count inliers
   Repeat
   Retain best model

8. Refinement
   Re-estimate the model using
   the final inlier set

OUTPUT:
   Verified matches
   Estimated geometric transformation
   Inlier mask

```

17 Applications and Engineering Requirements

Feature matching is used in many computer vision systems. The appropriate detector, descriptor, matcher, geometric model, hardware and accuracy requirement depend strongly on the application.

Application	How features are used	Typical accuracy requirement	Real-time / hardware considerations	Typical geometry
Image stitching	Detect and match features between overlapping images; estimate homography; warp and blend images.	Subpixel-to-few-pixel correspondence accuracy; robust matching across exposure/viewpoint changes.	Usually CPU/GPU; latency can be moderate because images are not always processed at video rate.	Homography

Application	How features are used	Typical accuracy requirement	Real-time / hardware considerations	Typical geometry
Panorama construction	Match features across multiple overlapping images; estimate pairwise transformations and globally optimize the camera graph.	High correspondence reliability; enough inliers for stable global alignment.	Desktop CPU/GPU; memory grows with number of images and descriptors.	Homography / rotation
Visual SLAM	Match features between consecutive frames and map points; reject outliers; estimate camera motion and update map.	High localization accuracy; often subpixel keypoint localization is valuable.	Strong real-time requirement, often 10–30+ Hz; embedded CPU/GPU or dedicated accelerators.	Epipolar geometry / pose model
Visual odometry	Track/match features across frames and estimate camera motion.	Low outlier rate and accurate pixel localization are important because errors accumulate over time.	Real-time; mobile/embedded CPU, GPU, or accelerator.	Essential / fundamental matrix
Object recognition / instance retrieval	Match local descriptors between a query image and a database image/object; geometric verification confirms object identity.	Moderate-to-high matching precision; robustness to viewpoint and illumination changes.	Offline retrieval can use large compute; online systems benefit from approximate NN search.	Homography when object is approximately planar
Image registration	Establish corresponding landmarks between images from different sensors, times or viewpoints and estimate transformation.	Can require subpixel or subpixel-equivalent accuracy, particularly in scientific and remote-sensing imaging.	Often offline or batch; CPU/GPU depending on image size.	Affine / projective / sensor-specific model
Remote sensing registration	Match landmarks between satellite or aerial images acquired at different times or sensors; refine geometric transformation.	Often high geometric accuracy; local distortions may require more than a global homography.	Large images favor tiled processing, GPU/parallel CPU; memory bandwidth is important.	Affine / projective / local warp
Medical image registration	Match anatomical landmarks or learned/local features between modalities or time points; geometric refinement aligns images.	Often very high accuracy; errors can have clinical consequences.	Usually workstation CPU/GPU; throughput less important than registration quality.	Affine / deformable models

Application	How features are used	Typical accuracy requirement	Real-time / hardware considerations	Typical geometry
Augmented reality	Match image features to a known target/map and estimate camera pose; continuously update tracking.	Low latency and stable localization; jitter must be small.	Strict real-time requirement; mobile CPU/GPU/NPU.	Homography / PnP / pose geometry
Industrial inspection	Match known reference features to current camera observations and identify displacement, defects or assembly errors.	High geometric accuracy; repeatability is important.	Controlled lighting; industrial PC, GPU or embedded processor depending on line speed.	Affine / projective / rigid
Robotics manipulation	Match object landmarks or scene features to estimate object/camera pose before grasping or manipulation.	Often millimetre-level task-space accuracy, translating to high image-space localization requirements.	Real-time edge compute; CPU/GPU/NPU depending on robot platform.	PnP / homography / 3D geometry
Drone navigation	Match visual landmarks between frames or against a map for localization and navigation.	Low latency, robust matching, and sufficient accuracy for stable pose estimation.	Severe power/thermal constraints; embedded CPU/GPU/NPU.	Essential / PnP / map geometry
Aerial mapping / SfM	Match features across many images; estimate camera poses and 3D structure.	High match precision and sufficient inlier count; errors propagate into reconstruction.	Usually offline workstation/server CPU/GPU; large-scale parallel processing useful.	Epipolar geometry
Visual localization	Match a query image against a database/map of reference images and geometrically verify candidate locations.	High precision needed to avoid incorrect localization.	Large databases benefit from approximate NN indexing and GPU/parallel retrieval.	Epipolar / pose / local 3D geometry

18 Application-Specific Design Choices

18.1 When Accuracy Is More Important Than Speed

For offline SfM, precision image registration, and scientific imaging:

SIFT + BF/accurate NN + ratio test + RANSAC

can be an excellent classical baseline.

The computational cost is acceptable because processing can be performed offline.

18.2 When Real-Time Operation Is Critical

For AR, robotics, visual odometry, and drones:

fast detector + compact descriptor + efficient matcher + geometric verification

is usually preferable.

Binary descriptors such as ORB can be attractive because Hamming distance is computationally inexpensive and can be implemented efficiently using bit operations. OpenCV specifically recommends Hamming-based matching for ORB/BRIEF/BRISK [1].

18.3 When the Database Is Very Large

Suppose there are millions of descriptors:

$$M \gg 10^6.$$

Exhaustive BF matching becomes increasingly expensive. Approximate nearest-neighbour indexing, vocabulary trees, inverted files, product quantization, or learned retrieval systems become more attractive.

Thus FLANN is best understood as one member of the broader family:

Approximate nearest-neighbour search.

19 Matching Accuracy versus Geometric Accuracy

A subtle but important point is that descriptor distance is not the same as geometric error.

A match may have a very small descriptor distance but still be geometrically incorrect:

$$D(\mathbf{d}_i, \mathbf{d}_j) \text{ small} \not\Rightarrow e_i \text{ small.}$$

Conversely, descriptor differences can occur for a correct correspondence because of:

- illumination change,
- blur,
- viewpoint change,
- scale change,
- affine distortion,
- sensor differences.

Therefore a robust system separates:

Appearance consistency	→ descriptor matching
Ambiguity rejection	→ ratio/cross-check
Geometric consistency	→ RANSAC
Final precision	→ geometric refinement

20 Subpixel Refinement After Matching

If very accurate registration is required, feature matching can be followed by geometric refinement.

Suppose a correspondence is initially:

$$\mathbf{x}_i = (x_i, y_i)$$

and

$$\mathbf{x}'_i = (x'_i, y'_i).$$

After RANSAC identifies reliable inliers, the geometric model can be refined using all inliers, for example by minimizing:

$$E(\theta) = \sum_{i \in \mathcal{I}} \|\mathbf{x}'_i - f(\mathbf{x}_i; \theta)\|_2^2,$$

where θ represents the transformation parameters.

For high-precision registration, one may further refine the image alignment directly using intensity-based optimization:

$$E(\theta) = \sum_{\mathbf{x}} [I_2(f(\mathbf{x}; \theta)) - I_1(\mathbf{x})]^2.$$

Thus the classical pipeline can become:

Detection $\rightarrow$ Description $\rightarrow$ Matching $\rightarrow$ RANSAC $\rightarrow$ Geometric optimization

This is particularly useful when the final application needs more precision than the raw keypoint detector can provide.

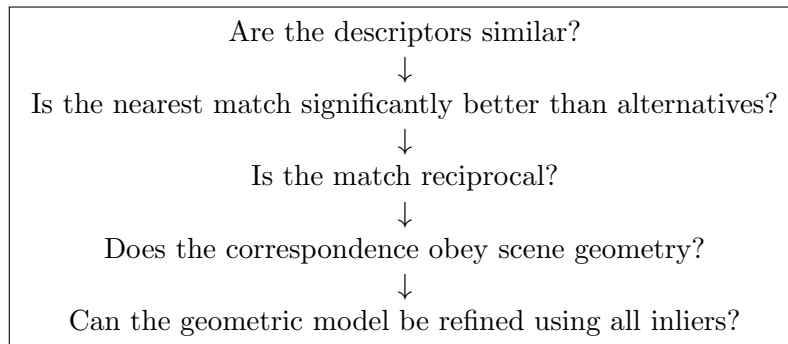
21 Common Implementation Caveats

1. **Use the correct distance.** Do not use Euclidean distance for binary ORB/BRIEF descriptors. Use Hamming distance.
2. **Do not rely only on descriptor distance.** A small descriptor distance does not guarantee a geometrically correct match.
3. **Do not blindly use the ratio threshold.** The optimal threshold depends on descriptor type and application.
4. **Cross-check and ratio test are different.** Cross-check tests reciprocity; ratio testing tests ambiguity against the second-best candidate.
5. **FLANN is approximate.** Its speed comes partly from avoiding exhaustive search. Increasing search effort generally improves nearest-neighbour quality at additional computational cost.
6. **Choose the geometric model correctly.** A homography is appropriate for planar scenes or particular camera motions, not arbitrary 3D scenes.
7. **RANSAC requires enough inliers.** If almost every descriptor match is incorrect, RANSAC may not recover the correct model.
8. **Avoid degenerate samples.** Collinear points, repeated patterns, and poorly distributed keypoints can make model estimation unstable.

9. **The RANSAC threshold is in image units.** A threshold of a few pixels should be interpreted relative to image resolution and localization uncertainty.
10. **Re-estimate after RANSAC.** Once the inlier set has been found, use all reliable inliers to obtain a better final transformation estimate.
11. **Benchmark on the target hardware.** The fastest algorithm on a desktop CPU may not be the fastest on an embedded processor, GPU, or NPU.

22 Summary

The complete classical feature matching problem can be understood as a sequence of increasingly stronger tests:



The corresponding algorithms are:

BF/FLANN → Ratio Test → Cross-Check → RANSAC → Geometric Refinement.
--

The fundamental lesson is:

Descriptor similarity provides a candidate correspondence; geometry provides evidence that the correspondence is real.

References

- [1] OpenCV Documentation, *Feature Matching*, OpenCV 4.x documentation. Available at: https://docs.opencv.org/4.13.0/dc/dc3/tutorial_py_matcher.html.
- [2] OpenCV Documentation, *Feature Matching with FLANN*, OpenCV documentation. Available at: https://docs.opencv.org/4.9.0/dc/dc3/tutorial_py_matcher.html.
- [3] D. G. Lowe, “Distinctive Image Features from Scale-Invariant Keypoints,” *International Journal of Computer Vision*, vol. 60, no. 2, pp. 91–110, 2004.
- [4] M. A. Fischler and R. C. Bolles, “Random Sample Consensus: A Paradigm for Model Fitting with Applications to Image Analysis and Automated Cartography,” *Communications of the ACM*, vol. 24, no. 6, pp. 381–395, 1981.